



COMPUTAÇÃO PARALELA

AULA 2



Prof. Leonardo Gomes



CONVERSA INICIAL

Nesta aula daremos início ao estudo da taxonomia de Flynn, que classifica sistemas computacionais multiprocessados. Discutiremos também as diferenças entre sistemas que operam em memória compartilhada e distribuída. Por fim, dando continuidade ao assunto relacionado à memória *cache*, nos aprofundaremos na questão da coerência entre múltiplas *caches*.

TEMA 1 – TAXONOMIA DE FLYNN: SISD

Com as barreiras físicas encontradas para a evolução dos *clocks* dos computadores modernos, a estratégia encontrada pela indústria para continuar fornecendo um poder de processamento ainda maior a cada geração de processadores foi justamente multiplicar o número de núcleos de processamento dentro de uma mesma máquina. Computadores multiprocessados e a possibilidade de combinar diferentes computadores para dividir uma ou múltiplas tarefas permitiu o surgimento da computação paralela em suas diferentes arquiteturas.

Múltiplos processadores permitem a execução de diversos processos e *threads* simultaneamente, aumentando o poder de processamento e diminuindo o tempo de resposta de determinados programas. Os multiprocessadores simétricos (SMP, do inglês, *symmetric multiprocessor*) são comumente encontrados nos computadores de uso geral domésticos e *clusters*.

Os *clusters* consistem em vários computadores independentes organizados de forma cooperativa. São desenvolvidos para trabalhar com grandes cargas de trabalho como servidores que atendem a um elevado número de requisições de diversos usuários, ou respondem a um único problema de grande complexidade.

Flynn, em 1966, propôs uma taxonomia que categoriza sistemas com capacidade de processamento paralelo adotada amplamente até hoje. Sua classificação possui quatro categorias principais baseadas no número de fluxos de controle (instruções) paralelos que um sistema consegue processar e de dados que consegue acessar. Na Figura 1 podemos visualizar a subdivisão dessas arquiteturas.



Figura 1 – Divisão proposta por Flynn dos sistemas multiprocessados

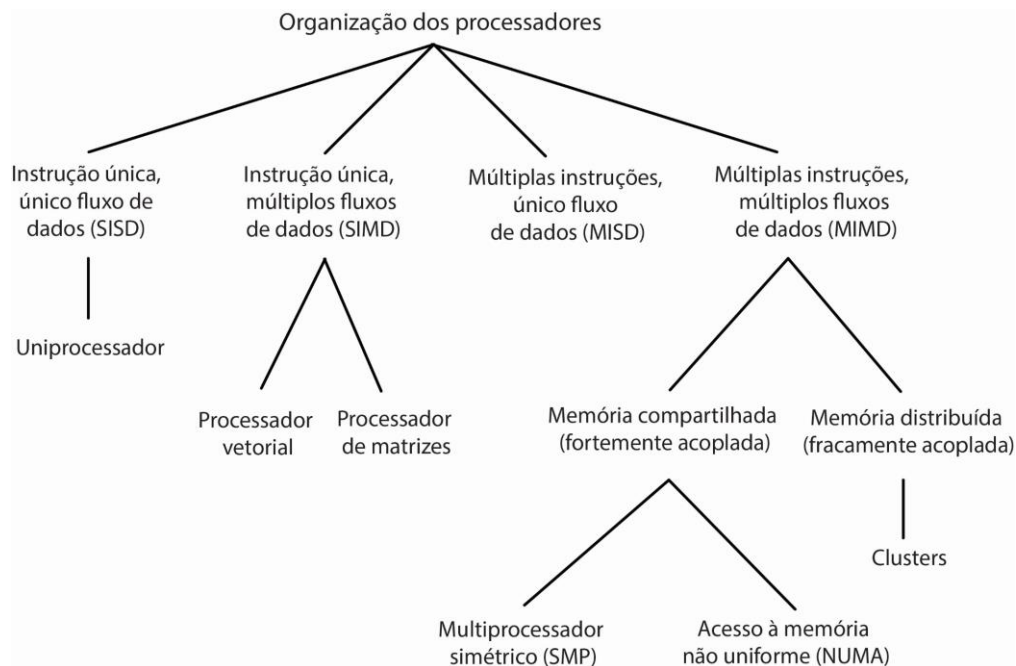
Fluxo de instruções	
um	muitos
Fluxo de dados	
um	SISD Computadores mono processados, lembrando Von Neumann tradicional
muitos	MISD Arquitetura utilizada em computadores com alto controle de falha. <i>pipelined</i>
	SIMD Processamento paralelo, processadores vetoriais, como nas placas gráficas
	MIMD Computadores multi processados.

A taxonomia de Flynn se tornou bastante popular academicamente devido à sua simplicidade. No entanto, vemos que algumas categorias se tornaram muito mais proeminentes – em meados dos anos 2000, a grande maioria dos computadores de alto desempenho trabalhava sobre o modelo MIMD. Por isso, além dessas quatro organizações, passou-se a ramificar um pouco mais a arquitetura MIMD com base na forma de acesso da memória pelos múltiplos processadores.

Temos então **memória distribuída** ou **compartilhada** e dentro da memória compartilhada segmentamos novamente em **processadores simétricos** de acesso **não uniforme**. Na Figura 2 é apresentada essa visão expandida sobre a taxonomia de Flynn. Discutiremos todas as arquiteturas em maiores detalhes na sequência.



Figura 2 – Representação da divisão proposta por Flynn dos sistemas multiprocessados e suas ramificações



Fonte: Stallings, 2017.

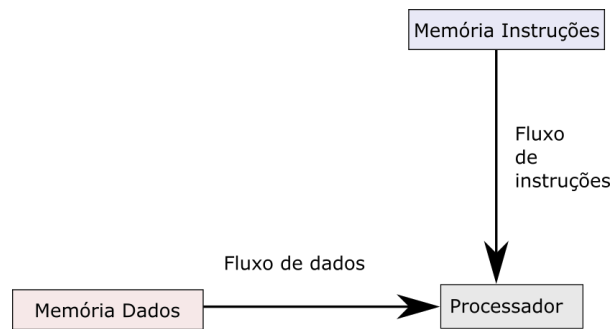
Na sequência, discutiremos a arquitetura SISD, a primeira das quatro organizações.

1.1 SISD

A organização de instrução única, único fluxo de dado (SISD, do inglês Single Instruction, Single Data) consiste em um processador único que opera uma única instrução por vez e executa armazenamento de dados em uma única memória. Instruções saem do módulo de memória para a unidade de controle, que decodifica as instruções e as envia para a unidade lógica aritmética, a qual, por sua vez, as processa e devolve para a memória.

Os uniprocessadores exemplificam bem esta categoria. Assemelham-se à visão tradicional da arquitetura de Von Neumann. Na Figura 3 temos uma representação desse sistema.

Figura 3 – Ilustração do sistema SISD



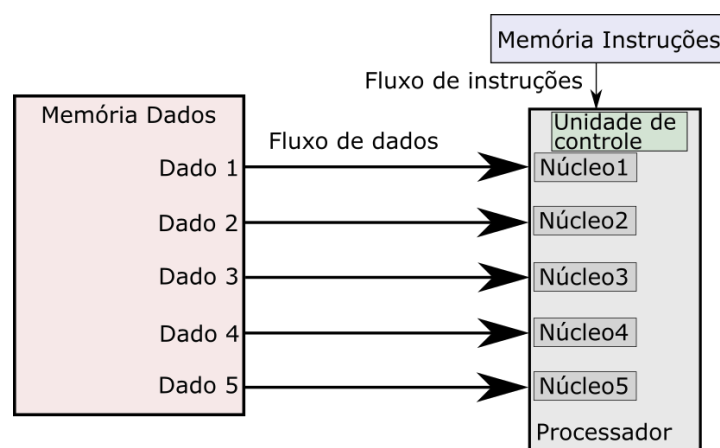
TEMA 2 – TAXONOMIA DE FLYNN: SIMD e MISD

Dando sequência às organizações propostas por Flynn, veremos as organizações SIMD e MISD.

2.1 SIMD

Os sistemas de instrução única, múltiplos dados (SIMD, do inglês *Single Instruction, Multiple Data*) possuem múltiplas unidades de processamento que operam as mesmas instruções simultaneamente. Cada unidade de processamento possui o próprio conjunto de dados, obtendo resultados diferentes para a mesma instrução. Os processadores de vetores e matrizes, tal qual as unidades de processamento gráfico (GPU – *Graphics Processing Unit*), se enquadram nesse grupo. Veja na Figura 4:

Figura 4 – Ilustração do sistema SIMD



Pela natureza multiprocessada da arquitetura SIMD, é muito adequada para operar sobre problemas computacionalmente pesados, como aqueles encontrados na computação científica. Problemas que trabalham com matrizes e vetores de grande escala em geral podem ser repartidos em muitas partes



menores, como submatrizes, para cada processador realizar uma mesma operação em um pedaço diferente do dado e remontá-lo depois.

A arquitetura conta com apenas uma unidade de controle e múltiplos processadores muito simples e organizados no formato de uma matriz com todos operando de forma síncrona. As placas gráficas operam dentro do mesmo paradigma SIMD, uma imagem pode ser entendida como uma matriz e as principais operações sobre ela são altamente paralelizáveis. As GPUs são desenvolvidas para acelerar todo o tipo de operação gráfica que no geral exigiria grande esforço computacional da CPU, desenvolvida para propósitos gerais. A indústria de jogos eletrônicos, por utilizar intenso processamento gráfico em tempo real, é uma das principais responsáveis pela demanda crescente de mercado por GPUs capazes de paralelizar milhares de núcleos de processamento em um único dispositivo.

Todo esse poder de processamento paralelo das placas gráficas dos dias de hoje incentivaram o surgimento de tecnologias capazes de programá-las para outros contextos que também se beneficiam de paralelismo. Dentre essas tecnologias capazes de utilizar GPUs como processadores de propósito geral, destacam-se duas APIs (do inglês *Application Programming Interface*, ou interface de programação): CUDA, desenvolvida pela empresa Nvidia, e OpenGL, desenvolvida originalmente pela Silicon Graphics nos anos 1990, e hoje com licença de *software* livre.

Ainda mais recentemente, temos as placas de processador voltadas para inteligência artificial (TPU, do inglês *Tensor Processing Unit*). Em termos estruturais, são bastante semelhantes as GPUs com diversos núcleos de processamento, no entanto são especialmente desenvolvidas para acelerar instruções envolvendo inteligência artificial e são planejadas para serem dispostas em servidores na nuvem.

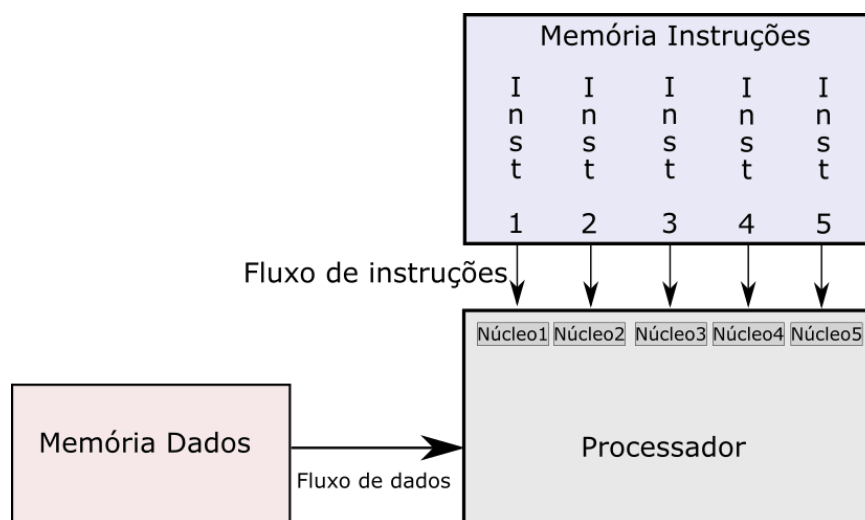
2.2 MISD

Na taxonomia Flynn, temos ainda os sistemas de múltiplas instruções, único dado (MISD, do inglês *Multiple Instruction, Single Data*), que consiste em uma sequência de dados que é transmitida para um conjunto de processadores, com cada processador executando uma sequência de instruções diferentes. Essa arquitetura não é muito usual, pois considera-se que o MIMD e o SIMD possuem maior flexibilidade quanto ao escalonamento de instruções paralelas.



O MISD se enquadra na execução de sistemas que operam com segmentação de instruções (*pipeline* do inglês). Uma grande vantagem é a possibilidade de redundância nas operações, tornando o sistema altamente resistente a falhas, e uma mesma operação pode ser realizada em múltiplos núcleos distintos, com o resultado validado por todos. Um exemplo proeminente que adota essa arquitetura são os computadores da estação de controle de voo dos ônibus espaciais da NASA, justamente pela alta tolerância a falhas. Na Figura 5 ilustramos essa estrutura.

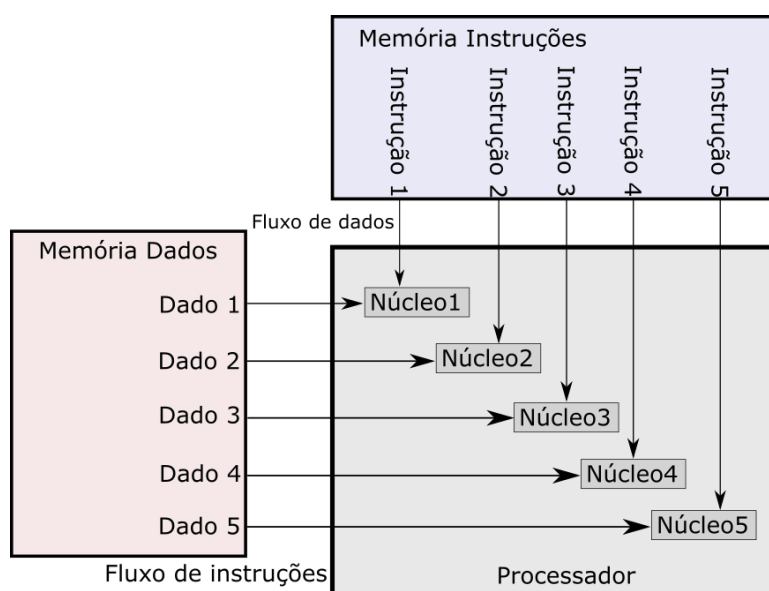
Figura 5 – Ilustração do sistema MISD



Por fim, dentro da taxonomia de Flynn, temos a arquitetura de múltiplas instruções, múltiplos dados (MIMD, do inglês *Multiple Instructions, Multiple Data*), que conta com múltiplos processadores executando de forma independente e assíncrona diferentes conjuntos de instruções em diferentes conjuntos de dados. Na categoria MIMD, podemos encontrar duas subcategorias de sistemas dependendo da forma de acesso da memória, a subcategoria de memória compartilhada e a de memória distribuída.

Os *clusters* e sistemas de acesso não uniformes à memória (NUMA, do inglês *Non-Uniform Memory Access*) se enquadram neste grupo. Ambos serão discutidos em detalhes a seguir.

Figura 6 – Ilustração do sistema MIMD



3.1 Memória distribuída

A memória distribuída considera que cada processador (ou conjunto deles) possui uma memória própria e executa de forma independente, e se comunicam por algum sistema externo ao barramento, tais como os *clusters*.

Por *cluster* entendemos um agrupamento de computadores completos, computadores capazes de funcionar por si só. Eles são conectados em alguma forma de rede com cada computador sendo entendido como um nó. Esse agrupamento de computadores dá ao usuário a ilusão de estar operando em um único computador.



Segundo Brewer (1997), podemos destacar quatro benefícios da organização em *cluster*:

- **Escalabilidade absoluta:** é possível combinar um número arbitrariamente grande de computadores nesse agrupamento, sendo que cada nó é um multiprocessador em si. O poder total de processamento de um *cluster* acaba sendo muito superior ao de uma máquina isolada.
- **Escalabilidade incremental:** um *cluster* permite incrementos pontuais, uma máquina por vez.
- **Alta disponibilidade:** o sistema em *cluster* acaba sendo altamente resistente a falhas pois se um nó for perdido, o sistema operacional, dependendo de suas configurações, pode conseguir tratar sem que ocorra nenhuma perda de serviço.
- **Maior custo/benefício no desempenho:** montar um computador de altíssimo desempenho por *cluster* acaba sendo significativamente mais barato do que montar uma máquina única equivalente.

A comunicação entre *clusters* costuma ser feita pela memória secundária, discos para os quais todos os nós possuem acesso. No geral, os discos são configurados em sistema RAID (*Redundant Array of Inexpensive Drivers*), de forma que a perda de um disco por falha não comprometa a integridade dos dados.

TEMA 4 – MEMÓRIA COMPARTILHADA

Em contrapartida da arquitetura de multiprocessadores com memória distribuída, temos a arquitetura de memória compartilhada. Como o nome sugere, os sistemas nessa arquitetura compartilham uma mesma memória com um mesmo sistema de endereçamento. Vamos discutir algumas estratégias para realizar esse compartilhamento e suas propriedades.

4.1 SMP

A forma mais comum dentro da arquitetura de memória compartilhada é o sistema de multiprocessadores simétricos (SMP, do inglês *symmetric multiprocessing*), que compartilha uma ou mais memórias por um único barramento ou algum outro mecanismo de interconexão. Nesse sistema, temos as seguintes características:



- Possui mais de um processador com poder de processamento, configurações e propriedades equivalentes;
- Compartilha uma mesma forma de interconexão com outros dispositivos, como um barramento, e utiliza a mesma memória com tempos de acesso aproximadamente semelhantes;
- Os processadores desempenham as mesmas funções de forma simétrica, recebendo meios de interação em todos os níveis de operação pelo sistema operacional.

Sistemas como o de um *cluster* não se enquadram nessa categoria, pois a interação entre os processadores não é direta, é feita por intermédio de algum outro elemento, como mensagens e compartilhamento de arquivos.

Até o início dos anos 2000, os computadores pessoais continham um único microprocessador de uso geral. No entanto, com a evolução da tecnologia que consistentemente reduz as dimensões e barateia os custos associados à produção de microprocessadores, os fabricantes introduzem sistemas SMP como regra – até mesmo *smartwatches* hoje possuem ao menos dois núcleos. Algumas vantagens da arquitetura SMP sobre uniprocessadores são:

- Desempenho, possibilidade de paralelizar atividades;
- Disponibilidade, a falha de um processador não impede a continuidade do funcionamento da máquina;
- Crescimento incremental, adicionar novos processadores pode incrementar ainda mais o poder de processamento na máquina;
- Escalabilidade, diferentes componentes em diferentes faixas de qualidade/preço podem ser ofertados por diferentes fornecedores.

Cada processador na arquitetura SMP é completamente independente, possuindo sua própria unidade de controle e unidade lógica aritmética, registradores, entre outros. O escalonamento de processos é resolvido integralmente pelo sistema operacional sem intervenção do usuário, ou seja, de forma completamente transparente, sem diferença dos uniprocessadores. Os processadores podem se comunicar pela memória principal ou mesmo por mensagens diretas. Geralmente a memória é organizada de forma que diferentes blocos podem ser acessados simultaneamente, minimizando tempo de espera.



4.2 NUMA

Outro modelo mais recente de multiprocessadores simétricos é o de acesso não uniforme à memória. Nesse modelo, cada processador leva um tempo diferente para acessar diferentes posições da memória, e também pode utilizar uma memória local mais rápida visando desafogar o barramento da memória compartilhada. Sistemas NUMA são frequentemente adotados visando servidores.

Da década de 1960 em diante, a velocidade de processamento ultrapassou a velocidade de acesso da memória, e essa diferença na velocidade continuou aumentando ao longo das décadas. Como efeito colateral dessa situação, os processadores se tornaram frequentemente ociosos enquanto aguardavam dados da memória, e ter múltiplos processadores aumenta ainda mais essa ociosidade, pois são mais processadores aguardando sua vez de utilizar o barramento.

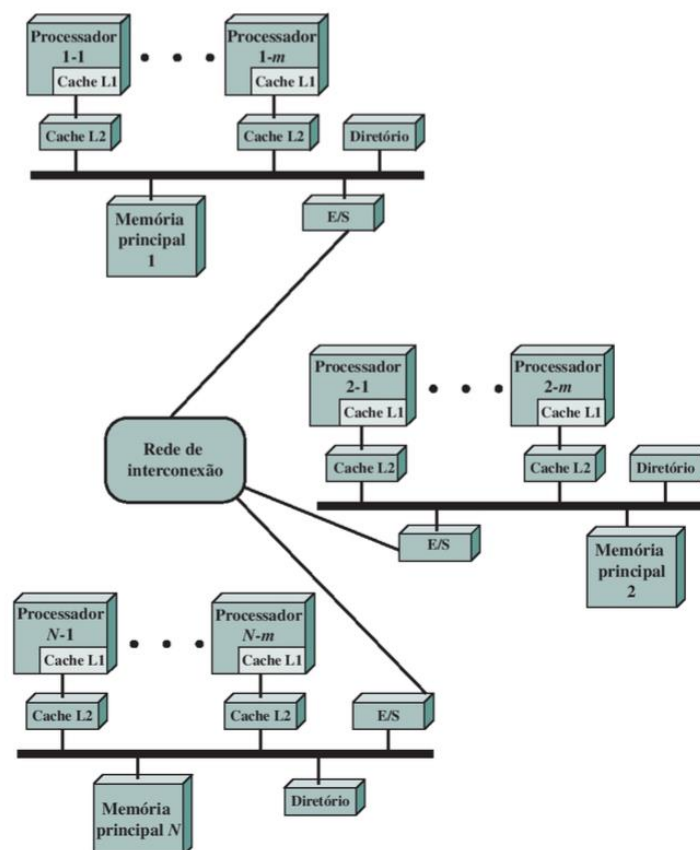
Otimizações da memória *cache* e o surgimento de sofisticados métodos de mapeamento auxiliaram nesse sentido e tornaram os processadores menos ociosos. Ainda assim existe um limite prático na quantidade de processadores que é viável ter em uma mesma máquina, por conta da competição pelo barramento, além dos processadores propriamente ditos – as próprias *caches* utilizam barramento para manter a coerência entre os dados de todos os processadores, tornando o barramento um gargalo muito limitante.

O desempenho da arquitetura SMP atinge seu máximo em torno dos 16 a 64 processadores, e o desempenho começa a cair por conta do gargalo com o barramento. As arquiteturas em *cluster*, que combinam diversas máquinas independentes com suas memórias próprias sem compartilhar um mesmo barramento, são uma resposta para essa limitação no crescimento do desempenho. Porém enfrentamos outro problema com os *clusters*, pois a gestão da memória e coerência deve ser feita via *software* em vez de *hardware*, consumindo processamento e recursos e necessitando do programador um *software* personalizado. A arquitetura NUMA busca um meio termo, oferecendo uma solução em *hardware* completamente transparente ao usuário e que permita combinar diversas estruturas SMP na forma de nós em uma rede única, atingindo mais de mil processadores organizados de formas independentes, cada um com sua própria estrutura de memória.



Cada nó possui sua própria memória, que, combinadas, formam a memória global do sistema. O endereçamento de memória é feito de forma a transparecer para cada processador que existe apenas uma única memória. Quando o um processador realiza um acesso à memória, inicialmente esse endereço é buscado dentro de suas próprias *caches* – caso falhe, o restante da memória principal será buscado; e caso esteja na própria memória, o acesso é feito utilizando somente o barramento local. Do contrário, uma requisição é feita de forma automática pela rede que liga os vários nós, e o dado é salvo na *cache* local exatamente como seria na arquitetura SMP. A Figura 7 ilustra essa arquitetura da forma que foi descrita.

Figura 7 – Ilustração representando o sistema NUMA



Fonte: Stallings, 2017.

TEMA 5 – COERÊNCIA DE CACHE

Visando desobstruir o barramento e evitar o gargalo de desempenho agravado por múltiplos processadores, as memórias *cache* se tornaram um conceito muito importante. Os processadores hoje contam com um,



dois ou até três níveis de memória *cache*, chamadas de L1, L2 e L3, tudo isso para evitar a necessidade de acessar o barramento. No entanto, como cada processador possui sua própria *cache*, pode acontecer que uma mesma região de memória esteja mapeada em diversas *caches* de diferentes processadores. Caso um desses processadores modifique o dado de uma região de memória na sua *cache* particular, essa mudança deve ser refletida em todas as *caches* em que o dado aparece, a fim de que não haja inconsistências dos dados.

As *caches* possuem duas políticas diferentes referentes à escrita em memória. Uma é chamada *write back*, na qual as escritas são feitas somente na *cache* e a memória principal é atualizada somente quando aquela região de memória retorna para a memória principal. E existe também *write through*, na qual todas as escritas são feitas tanto na *cache* quanto na memória principal. Independentemente da política de escrita, o problema da inconsistência pode ocorrer, pois o dado inconsistente fica localizado nas *caches* dos processadores que não realizaram a escrita. Temos diferentes alternativas para resolver essa questão, em *software* e em *hardware*. Vamos conferir a seguir.

5.1 Alternativas por *software*

A coerência pode ser resolvida via *software* pelo sistema operacional e o compilador do programa. Durante compilação, o programa pode identificar quais regiões são as mais críticas que irão gerar problemas com *cache* e marca, de forma que simplesmente não sejam gravadas em *cache* e apenas na memória principal – essa solução acaba sendo ineficiente pois não otimiza ao máximo a *cache*. Algumas soluções um pouco mais robustas propõem compiladores que analisam o código de forma a identificar o período de tempo em que determinada região da memória será problemática e marca adequadamente no código para que o sistema operacional saiba quando bloquear e liberar a *cache* de armazenar determinada região.

O fato de a coerência ser resolvida por *software* tem a vantagem clara de dispensar a necessidade de criar ainda mais circuitos. Além disso, transfere a resolução do problema para tempo de compilação ao invés de execução. No entanto, como vimos, as soluções envolvem subutilizar *cache*.



5.2 Alternativas por *hardware*

As alternativas em *hardware* de forma geral conseguem entregar um desempenho melhor por tratar o problema apenas quando as inconsistências acontecem. Outra vantagem é que fazem isso de forma completamente transparente ao programador, que não precisa pensar nessa questão. As estratégias de *hardware* são divididas em duas categorias, protocolos de diretório e protocolos de monitoramento.

Protocolos de diretório armazenam na memória principal a informação de quais blocos de memória estão sendo armazenados por quais processadores. Quando um processador deseja escrever em um bloco de memória, ele solicita o acesso para o diretório, que irá invalidar a informação de todos os demais processadores, garantindo que nenhum processador utilize uma informação inválida de sua *cache* particular. Quando um processador solicitar acesso para um bloco de memória reservado, o diretório irá solicitar ao processador que fez a reserva para reescrever o dado na memória principal.

A desvantagem de uma estratégia baseada em protocolos de diretório é o uso intensivo do barramento para manutenção do controle. Isso é desvantajoso para um esquema com barramento único, porém eficiente para um sistema com múltiplos barramentos ou formas de interconexão.

Protocolos de monitoramento distribuem a incumbência de manutenção da coerência das *caches* entre todos os processadores, diferentemente dos protocolos de diretório que centralizam essa atividade. A estratégia é que, ao escrever na memória *cache* em um bloco de memória compartilhado com vários outros processadores, essa informação deve ser propagada para todos os demais processadores. Isso é vantajoso em sistemas que possuem um único barramento, pois como todos os processadores estão ligados nele, um único acesso é necessário para atualizar todos simultaneamente.

5.3 Protocolo MESI

O protocolo MESI, desenvolvido na Universidade de Illinois (sendo também conhecido como protocolo de Illinois) é um dos mais populares protocolos de coerência de *cache* e de memória atualmente. Ele adiciona dois bits de controle para cada dado, ou linha, salvo na memória *cache*.



Com esses dois bits, é possível marcar quatro estados distintos:

- **Modificada:** a linha da *cache* foi modificada pelo próprio processador, ou seja, está com um novo valor e a memória principal possui um valor desatualizado.
- **Exclusiva:** a linha da *cache* é igual àquela na memória principal, não sofreu modificações e não está presente em nenhuma outra *cache* de outro processador.
- **Compartilhada:** a linha da *cache* é igual àquela na memória principal e pode estar presente em outra *cache* de outro processador.
- **Inválida:** a linha da *cache* não contém dados válidos; outro processador modificou aquele valor e qualquer tentativa de leitura dessa linha conta como falha automática.

Esses quatro estados ajudam a controlar o comportamento das *caches* nas diversas situações que acessos de leitura e escrita de um bloco de memória podem gerar. Vejamos em maiores detalhes:

- **Falha na leitura:** caso determinado processador tente realizar uma **leitura** e **não encontre** o dado em sua *cache*, ele deve buscar essa informação na memória principal. Caso outras *caches* possuam cópia exclusiva ou compartilhada, todas passam a ser compartilhadas. Caso outra *cache* seja modificada, ela deve atualizar a memória principal e todas passam a ser compartilhadas também.
- **Acerto na leitura:** caso determinado processador tente realizar uma **leitura** e **encontre** o dado na sua *cache*, a leitura é feita e nenhuma mudança de estado acontece.
- **Falha na escrita:** caso determinado processador tente realizar uma operação de **escrita** e **não encontre** o dado em sua *cache*, o processador vai até a memória principal realizar sua escrita. Nessa situação, o processador envia um sinal para todos as *caches* indicando sua intenção de alterar aquele bloco de memória. Se outro processador tiver aquela linha marcada como modificada, ele atualiza primeiro o dado na memória principal e marca sua própria linha como inválida. Demais processadores que compartilhem a linha também marcam como inválida, e o processador que fez a requisição de escrita lê a linha e a modifica em sua *cache*, marcando-a como modificada.



- **Acerto na escrita:** caso determinado processador tente realizar uma operação de **escrita** e **encontre** o dado em sua *cache*, se a linha estiver marcada como exclusiva ou modificada, ela se torna continua modificada. Caso seja compartilhada, os demais processadores são comunicados pelo barramento para invalidar suas linhas, e o processador que iniciou a escrita marca sua linha como modificada.

FINALIZANDO

Comentamos, neste encontro, as arquiteturas paralelas propriamente ditas e discutimos em detalhes as quatro abordagens da taxonomia de Flynn: SISD e seu sistema monoprocessado tradicional; SIMD, que paraleliza uma mesma instrução para diferentes conjuntos de dados; MISD, que executa diferentes instruções em um mesmo dado; e a MIMD, que distribui múltiplas instruções em múltiplos núcleos de processamento – arquitetura mais comumente vigente nos computadores multiprocessados. Vimos também as diferenças entre as arquiteturas que adotam memória compartilhada e as que utilizam memória distribuída dentro da arquitetura MIMD. Por fim, falamos de como é realizada a coerência entre os dados que são representados em múltiplas memórias *cache*.

Com esses estudos, temos uma boa visão geral sobre os elementos que compõem os sistemas paralelos e suas finalidades. Em breve, discutiremos como é feita a codificação em sistemas paralelos.



REFERÊNCIAS

BREWER, E. **Clustering**: multiply and conquer. Data Communications, July 1997.

STALLINGS, W. **Arquitetura e organização de computadores**. 10. ed. São Paulo, 2017.